

İçindekiler

1. Dönem Projelerinde Olması Gereken Özellikler.....	1
2. Bonus Puan Verilecek Özellikler	2

1. Dönem Projelerinde Olması Gereken Asgari Özellikler

1. Veritabanı işlemleri, kullanıcı hikâyeleri (user stories) ve kabul kriterleri (acceptance criterias) doğrultusunda ihtiyaç oldukça veritabanı tasarımı iteratif olarak yapılacak ve her artımla kademeli olarak gerçekleştirilecektir.
2. Veritabanı işlemleri **ORM (Object–Relational Mapping)** ve **Repository Pattern** kullanılarak geliştirilecektir; böylece veri erişim kodu iş mantığından ayrılacak, daha sürdürülebilir/okunabilir bir katmanlı yapı ve daha kolay test edilebilir bir tasarım hedeflenecektir. Tüm veritabanı etkileşimleri yalnızca ORM üzerinden yapılacak, ham SQL komutları doğrudan kullanılmayacaktır; şema değişiklikleri migration ile yönetilecek ve tablo ilişkileri (1-N, N-N vb.) ORM üzerinde doğru şekilde modelleneyecektir. Ayrıca CRUD, filtreleme, listeleme ve arama gibi tüm veri erişim işlemleri Repository sınıfları üzerinden yürütülecek, doğrudan ORM çağırısı yapılmayacaktır. Repository'ler ile veri erişim metotları tek yerde toplanarak kod tekrarı azaltılacaktır.
3. Dönem projeleri proje yönetim aracı **Jira** kullanılarak **Kanban Çevik (Agile) Yazılım Geliştirme** yaklaşımı ile yönetilecektir.
 - a. Proje notlandırmaları Çevik Yazılım Geliştirme yönteminin genel prensiplerine ve özelinde Kanban yönteminin prensiplerine ne kadar uyulduğuna göre yapılacaktır.
 - b. Jira Kanban'da tanımlanan işlerin (task, story vb.) Git&GitHub'ta ilgili branch'lere karşılık gelmesi sağlanacaktır. Tasklar/Story'ler Kanban'da farklı aşamalardan (To Do, In Progress, Done vb.) geçerken Git/GitHub'da karşılık gelen işlemler (branch oluşturma, merge, vb.) yapılacak. Örneğin bir task Kanban'da To Do'dan In Progress'e alındığında bu task için main branch'ten bir feature branch oluşturulacaktır. Bu şekilde, kod geliştirmenin ilerlemesi ile proje yönetimi arasında izlenebilir bir uyum sağlanacaktır.
 - i. Bunun için linkerdeki ([link1](#), [link2](#), [link3](#), [link4](#)) kaynaklardan veya başka kaynaklardan faydalanılarak Jira ile Git entegrasyonu sağlanacaktır. GitHub repo'daki proje ile Jira entegrasyon sağlandıktan sonra Jira tarafından bir story/task kartı için Git branch başlatılabilecek, ilgili task/story için kod geliştirme tamamlandıktan sonra pull request başlatılabilecek, pull request tamamlandıktan sonra yapılan merge/commit işlemleri story/task kartının içinde görülebilecektir. Hangi task/story için hangi kod değişikliklerinin yapıldığı izlenebilecektir.
4. Projelerin geliştirme ve takip süreçleri dönem boyunca **Git ve GitHub** üzerinden yürütülecektir.
 - a. Projeler GitHub repository üzerinde yönetilecektir. **Projelerin tüm raporları ve kodları dönem sonunda Turnitin'e yükleneceğinden Turnitin'de yanıltıcı %100 benzerlik çıkmasını önlemek için GitHub reposu Private olmak zorundadır. Repo**

erişimi yalnızca takım üyeleri ile proje kontrolü amacıyla asistan hocalara paylaşılacaktır.

- b. GitHub depoda proje için README dosyası olacak. README içinde olması gerekenlerle ilgili örnek bir kılavuz için [tıklayınız](#).
 - c. Projenin kolayca gezinilebilmesi için GitHub depo mantıksal bir yapıyla düzenlenecektir (örneğin kaynak kodu, belgeler, testler için ayrı klasörler gibi).
 - d. Git commitler'i yapılan işle ilgili açıklayıcı mesajlar ile yapılacaktır.
 - e. GitHub reposu master/main branch ve feature branch'ler olmak üzere iki kategoriden oluşacaktır. Main branch içerisinde bulunan kaynak kodu her zaman için projenin yayınlanmaya hazır hali olacaktır.
 - f. Branch yönetimi örnekteki ([link1](#)) gibi GitHub Workflow ile yapılacaktır. Yeni bir feature veya düzeltme başlatıldığında main branch'ten yeni bir feature branch oluşturulacak. Öğrenciler, işbirliğine dayalı kod incelemesi (code review), geri bildirimin dahil edilmesi ve conflict'lerin etkili bir şekilde çözülmesi için pull request kullanacaklardır. Yeni feature için kodlama tamamlandıktan sonra lokalden GitHub repoya push edilecek pull request (PR) ile code review süreci başlatılacaktır. PR için en az 1 takım üyesinin incelemesi/Onayı (approval) alınmadan main branch'e merge yapılmayacaktır. Review yapanlar; buldukları hataları, önerileri veya olumlu geri bildirimleri belirtecekler. Örneğin, bir değişiklik talep edebilecek veya bir koda onay verebilecektir. PR review tamamlandığında PR için atanan kişi/kişiler onay verecek ardından main branch'e merge edilecek ve ilgili feature branch silinecektir.
5. Projelerin yapısına uygun en az bir **tasarım deseni** (design pattern) ve bir **mimari desen** (architectural pattern) kullanılacaktır.
 6. Projeler **Unit Testler** yazılarak geliştirilecektir.
 7. Projeler takım odaklı çalışma prensiplerine göre yapılacaktır.

2. Bonus Puan Verilecek Özellikler

1. **Projeler bir bulut ortamına CI/CD pipeline ile otomatik olarak deploy edilerek gerçekleştirilmesi (Bu özellik eklendiğinde Final Notuna +15 puan eklenecektir.):**
 - a. Dönem projeleri; Render, Railway vb. bir bulut ortamına canlı (production) olarak deploy edilmiş şekilde gerçekleştirilecektir ve GitHub Actions kullanılarak oluşturulmuş bir CI/CD pipeline ile otomatik olarak yayınlanacaktır.
 - b. Pull Request açıldığında otomatik olarak build, lint ve test aşamaları çalıştırılacaktır. main branch'e yapılan merge işlemi sonrasında uygulama otomatik olarak deployment ortamına gönderilecek ve manuel deploy işlemi yapılmayacaktır. Süreç tamamen otomasyon mantığıyla kurgulanacaktır.
 - c. Veritabanı olması gereken projelerde, veritabanı da bulut ortamında barındırılacaktır; bu amaçla Render PostgreSQL, Railway PostgreSQL/MySQL, Supabase, Neon veya MongoDB Atlas gibi ücretsiz veritabanı hizmetlerinden yararlanabilirsiniz.
 - d. Ortam değişkenleri ve hassas bilgiler (örneğin veritabanı bağlantı bilgileri ve API anahtarları) kod içerisinde tutulmayacak, GitHub Secrets üzerinden güvenli biçimde yönetilecektir.
 - e. Proje tesliminde canlı sistemin erişilebilir URL'si, ilgili GitHub repository bağlantısı ve CI/CD pipeline süreci proje teslim raporunda sunulacaktır.
 - f. Linklerdeki kaynaklardan veya bunlara benzer başka kaynaklardan faydalanılabilir: [link1](#), [link2](#), [link3](#).
2. En az 1 adet yapay zekâ API'si ile entegre olunması ve bu API üzerinden alınan çıktının uygulama içinde anlamlı bir şekilde kullanılarak görüntülenmesi (ör. OpenAI API, Gemini API,

Claude API, Hugging Face Inference API; kullanım senaryosu olarak metin özetleme veya sınıflandırma, duygu analizi, soru-cevap, içerik önerisi, belge arama, görsel açıklama vb.).”

3. Branch yönetiminin GitHub Flow yerine Git Flow branch hiyerarşisi (feature branch, develop branch, release branch, main branch [link1](#), [link2-5.Bölüm Git Flow](#)) ile sürüm (release) yönetimi şeklinde yapılması.